

自作PCパーツECサイト 詳細設計書（レビュー版）

版 0.9 / 作成日 2026年9月25日 / 対象 初期版・公開ポートフォリオ

要件の正本 [output/pdf/自作PCパーツECサイト_要件定義書_レビュー版.pdf](#)

継承する設計 [output/pdf/自作PCパーツECサイト_基本設計書_レビュー版.pdf](#)

状態 技術設計レビュー用。送料・公開デモの個人情報方針は2026年9月25日に利用者確認済み。外部サービスの有料利用のみ事前承認が必要。
製品実装・DB変更は未着手。

1. 設計境界と追跡

初期版は8カテゴリを扱う国内向け模擬ECである。実販売・実課金・実発送はしない。Vercel上のNext.js App Router、React、TypeScript、CSS Modules、Zod、Supabase Postgres/Auth/Storage/Cron、Stripe Checkoutテスト環境、GitHub Actionsを基本設計どおり採用する。公開デモではメールを実送信し、SMSは模擬通知で6桁コードの入力・照合を体験する。ユーザー向け取り置き、ゲスト注文、構成保存、外部商品価格連携は将来拡張である。

要件IDの主な対応は、商品探索=CAT、互換性=CMP、会員=ACC、在庫=INV、注文・送料・決済=ORD、管理=ADM、異常系=DEM、保護=SEC。送料と公開デモの個人情報方針は利用者の確認を受けて本書で確定した。有料サービスを利用する判断は利用開始前に別途確認する。

2. UIデザイン規則とレスポンス

2.1 色・文字・共通部品

● 主背景

色: #F7F9FC / 使い方: 全画面の背景

● サーフエス

色: #FFFFFF / 使い方: カード、フォーム、表

● 本文

色: #172033 / 使い方: 通常本文。白背景に対して十分な明度差を確保

● 補助文字

色: #4B596E / 使い方: 説明、仕様ラベル

● 主操作

色: #155EEF / 使い方: 主要ボタン、リンク、選択状態

● 主操作文字

色: #FFFFFF / 使い方: 主操作上の文字

● 注意

色: #9A5B00 / #FF4D6 / 使い方: 互換性不一致、送料の概算など

● 危険

色: #B42318 / #FEF3F2 / 使い方: 決済失敗、入力エラー、在庫不足

● 成功

色: #067647 / #ECFDF3 / 使い方: 互換性一致、注文完了

● 境界

色: #CBD5E1 / 使い方: 枠線、区切り

システムフォントを基礎に日本語可読性を優先する。本文16px、補助14px以上、見出し24~32px、行間1.5、操作領域44px以上、フォーカスリング2pxとする。8px刻みの余白、カード角8px、価格・在庫・仕様をカード内の同じ位置に置く。警告と成功はアイコン・ラベル・本文も併記し、色だけに依存しない。画像には意味のある代替テキストを付ける。WCAG 2.2 AAを検証基準とする（本文4.5:1、UI部品3:1、キーボード操作とエラー説明）。

2.2 ブレークポイントと変化

● スマートフォン 0-767px

レイアウト: 1列。検索はヘッダー直下、カテゴリは横スクロール可能なリスト、絞り込みは開閉パネル。商品カード1列、価格と購入操作を見やすく配置。管理表はカード化。

● タブレット 768-1199px

レイアウト: 商品カード2列、詳細は画像と情報の2列。絞り込みは開閉パネル。ヘッダーの補助メニューは折りたたむ。管理一覧は重要列を表示し詳細へ移動。

● PC 1200px以上

レイアウト: 最大幅1280px。商品カード3〜4列、一覧の左に幅260pxの絞り込み。詳細は画像・仕様・購入欄の2〜3領域。管理画面はサイドナビと表。

画面幅320pxでも横方向のページスクロールを生じさせない。フォームの2列入力は767px以下で1列にする。注文確認の金額サマリーはPCで右列、モバイルで明細の後に配置し、決済ボタンを常に合計直後に置く。構成確認はPCでカテゴリごとのカード2列、モバイルは1列。固定表示する場合でもフッター・エラー・キーボードを覆わない。すべての画面でブラウザ拡大200%を確認する。

2.3 共通UIと状態

ヘッダーはロゴ、検索、8カテゴリ、用途別、構成確認、カート件数、会員操作。フッターは模擬販売の説明、プライバシー案内、ヤマト運輸公式料金ページ、問い合わせ先。パンくずを詳細・カテゴリ・管理配下に表示する。読み込み中は寸法の近いスケルトン、失敗時は「原因／次にできる操作」、空状態は推奨導線を出す。確認モダルは破棄操作に限り、互換性不一致では使わない。トーストは保存成功などの補助通知であり、入力エラーや決済結果は画面内にも残す。

3. 画面詳細

画面IDをテストと遷移の共通キーにする。入力値はURLクエリまたはフォームに保持し、検証失敗で失わない。ログイン要求時は同一サイト内の安全な戻り先だけを保持する。

● S01 /

表示・操作・入力: 用途別（ゲーム・普段使い・動画編集）、8カテゴリ、検索、初心者向け説明。 / **エラー／空状態と遷移:** 掲載商品がない用途はカテゴリ一覧へ。検索→S03、カテゴリ→S02。

● S02 /categories/[slug]

表示・操作・入力: カテゴリ説明、仕様別フィルタ、価格、在庫、メーカー、並べ替え、ページ送り。 / **エラー／空状態と遷移:** 不正slugは404。0件なら条件解除。商品→S04。

● S03 /search

表示・操作・入力: キーワード、カテゴリ、用途、価格帯、メーカー、カテゴリ固有の仕様。条件はURL共有可能。 / **エラー／空状態と遷移:** 無入力は全公開商品。0件は条件を残し解除ボタン。入力長超過は説明。商品→S04。

● S04 /products/[slug]

表示・操作・入力: 画像、型番、税込価格、在庫状態、仕様、初心者向け選び方、数量1〜10、カート追加、構成に追加。 / **エラー／空状態と遷移:** 非公開・存在しない場合404。販売可能0は追加不可で「在庫切れ」。追加中に不足なら数量再指定を案内。→S05/S06。

● S05 /build

表示・操作・入力: 8カテゴリの選択カード、5判定の結果、比較値、理由、適合商品リンク、カート追加。選択はURLまたはブラウザ内の一時状態。 / **エラー／空状態と遷移:** 未選択は選択を促す。仕様不足は「判定できません」と不足項目を表示。不一致でも追加可能。→S02/S03/S04/S06。

● S06 /cart

表示・操作・入力: 明細、数量変更・削除、商品小計、送料概算、税込合計見込み、会員のみ購入へ。 / **エラー／空状態と遷移:** 空なら商品探索へ。価格変更・在庫不足・販売終了を行単位で表示し再確認。未ログイン→S07、会員→S12。

● S07 /login

表示・操作・入力: メール・パスワード、Google、再設定、登録。 / **エラー／空状態と遷移:** 認証失敗はアカウント有無を推測させない文言。成功→安全な戻り先かS15。

● S08 /register

表示・操作・入力: メール、パスワード、確認、規約・個人情報案内、登録。 / **エラー／空状態と遷移:** 形式・強度・一致を欄下に表示。登録後→S09。

● S09 /verify

表示・操作・入力: メール確認案内、再送、6桁模擬SMS入力・再発行。公開デモで実SMSではないと明示。 / **エラー／空状態と遷移:** 期限切れ・試行超過は再発行を案内。両確認後→S15または購入復帰。

● S10 /reset-password

表示・操作・入力: メール送信、リンク経由の新パスワード、6桁SMS照合。 / **エラー／空状態と遷移:** リンク・コード期限切れ、再送制限を表示。完了→S07。

● S11 /account/addresses

表示・操作・入力: 住所一覧と登録・編集・削除。氏名、郵便番号、都道府県、市区町村、番地、建物、既定フラグ。架空情報の入力を案内。 / **エラー／空状態と遷移:** 空なら新規登録。形式エラーを欄下に表示。編集後は元画面へ戻る。削除時は対象を明示して確認。

● S12 /checkout/address

表示・操作・入力: ログイン済み限定。保存先の選択または追加、注文用住所確認。 / **エラー／空状態と遷移:** 住所なし→入力。国内住所形式エラー、カート空・売切れはS06へ。→S13。

● S13 /checkout/review

表示・操作・入力: 商品・住所・確定送料内訳・税額・合計・互換性警告・模擬決済注記を表示。決済へ進む操作。 / **エラー／空状態と遷移:** 再計算差異は更新値を表示し再確認。在庫不足は該当商品とS06への導線。送料算定不能は決済不可で理由表示。→Stripe。

● S14 /checkout/result

表示・操作・入力: 決済中／成功／失敗／期限切れをサーバー取得で表示。 / **エラー／空状態と遷移:** Webhook未着は「確認中」と再読み込み。失敗は再試行またはS06。成功→S17。戻りURLのみで成功を表示しない。

● S15 /account

表示・操作・入力: 会員概要、配送先、注文履歴、ログアウト、アカウント削除。30日でデモ個人情報を削除する説明。 / **エラー／空状態と遷移:** 未ログイン→S07。注文なしは商品探索へ。削除は再認証・確認後に実行しS01へ。

● S16 /account/orders

表示・操作・入力: 自分の注文一覧、日付・金額・状態。 / **エラー／空状態と遷移:** 空なら探索へ。注文→S17。

● S17 /account/orders/[id]

表示・操作・入力: 注文時の商品・住所・送料・税・決済状態のスナップショット。 / **エラー／空状態と遷移:** 他人の注文は404相当。未確定は確認中、失敗は再試行導線。

● A01 /admin

表示・操作・入力: 管理者ダッシュボード、公開商品・在庫不足・注文・通知障害の件数。 / **エラー／空状態と遷移:** 非管理者403。件数0は「対象なし」。

● A02 /admin/products

表示・操作・入力: 商品一覧、新規、検索、公開状態。編集画面は8カテゴリ固有仕様、価格、重量・梱包寸法、画像。 / **エラー／空状態と遷移:** 必須仕様不足は下書き保存可、公開時は検証。保存失敗は入力保持。

● A03 /admin/inventory

表示・操作・入力: 商品別の実在庫・引当中・販売可能数、調整理由・数量入力。 / **エラー／空状態と遷移:** 負数や引当数を下回る調整は拒否。競合時は最新値表示。

● A04 /admin/orders

表示・操作・入力: 注文一覧、状態・決済試行・監査の参照。 / **エラー／空状態と遷移:** 注文なしは対象なし。決済状態は手編集しない。

● A05 /admin/settings/shipping

表示・操作・入力: 発送元、通常送料940円、関値1万円、重量物の公式料金表・根拠URL・確認日・規則版。 / **エラー／空状態と遷移:** 料金表データ欠損では正式見積不可。設定変更は保存前に差分・監査。

● A06 /admin/settings/email

表示・操作・入力: SMTPホスト・ポート・送信元・TLS・秘密入力欄、接続試験。 / **エラー／空状態と遷移:** 秘密は再表示しない。試験失敗は原因を秘密抜きで表示。

管理画面は一般ECと異なるヘッダー・サイドナビを使う。管理者でない利用者に管理リンクを出さず、URL直打ちでも拒否する。スマートフォンのA02/A03/A04は表をカードへ変換し、行全体の編集操作を維持する。

4. データ設計の共通規則

PostgreSQLを使用。主キーはuuid、日時はtimestamp with time zone、金額は税込円のinteger（0以上）、重量はintegerグラム、長さはintegerミリメートル、数量はinteger（原則1以上）。表示時のみ円記号を付ける。created_atとupdated_atを保持し、更新日時はDBで更新する。外部に出す連番IDは作らず、UUIDはアクセス権の代替にしない。deleted_atは商品などの論理非公開に限定し、注文履歴はスナップショットを保存する。列举値はDBのCHECKまたは専用enumで許容値を固定する。すべての外部キーは索引を確認し、検索の主要条件は複合索引を付ける。

4.1 会員・認証

● profiles

主要カラム（型・制約）: `user_id` uuid PK FK `auth.users`, `display_name` text, `created_at` timestampz, `updated_at` timestampz。電話番号は公開デモで収集しない。 / 関連・索引: 本人1対1。

● admin_memberships

主要カラム（型・制約）: `user_id` uuid PK FK `auth.users`, `granted_by` uuid FK `auth.users`, `granted_at` timestampz, `revoked_at` timestampz null。 / 関連・索引: 有効権限に部分索引。本人更新不可。

● addresses

主要カラム（型・制約）: `id` uuid PK, `user_id` uuid FK, `recipient_name` text not null, `postal_code` char(7), `prefecture_code` smallint CHECK 1..47, `city` text, `street` text, `building` text null, `is_default` boolean, `created_at/updated_at`。 / 関連・索引: (`user_id`, `created_at`)、ユーザーごと既定住所1件の部分ユニーク索引。

● sms_challenges

主要カラム（型・制約）: `id` uuid PK, `user_id` uuid FK null, `purpose` text CHECK(register,reset), `target_session_hash` text, `code_hash` text, `expires_at` timestampz, `attempts` smallint CHECK 0..5, `verified_at` timestampz null, `delivery_mode` text CHECK(mock,real), `created_at`。 / 関連・索引: (`user_id`, `purpose`, `created_at` desc)。コード平文・電話番号は保存しない。

4.2 商品・仕様・画像

● categories

主要カラム（型・制約）: `id` uuid PK, `slug` text unique, `name` text, `sort_order` smallint。初期値8件。 / 関連・索引: `slug`一意。

● products

主要カラム（型・制約）: `id` uuid PK, `category_id` uuid FK, `slug` text unique, `sku` text unique, `name` text, `brand` text, `description` text, `beginner_note` text, `price_tax_included_yen` integer CHECK >=0, `tax_rate_basis_points` smallint DEFAULT 1000, `status` text CHECK(draft,published,hidden), `weight_g` integer CHECK >0, `pack_length_mm/width_mm/height_mm` integer CHECK >0, `created_at/updated_at`。 / 関連・索引: (`category_id`, `status`, `price_tax_included_yen`, `id`), (`status`, `brand`), 検索用tsvectorまたはpg_trgm索引。公開可否と送料計算に必要な項目を公開時検証。

● product_use_cases

主要カラム（型・制約）: `product_id` uuid FK, `use_case` text CHECK(gaming,daily,editing), 複合PK。 / 関連・索引: (`use_case`, `product_id`)。

● product_images

主要カラム（型・制約）: `id` uuid PK, `product_id` uuid FK, `storage_path` text unique, `alt_text` text, `sort_order` smallint, `created_at`。 / 関連・索引: (`product_id`, `sort_order`)。公開画像のみ配信。

● cpu_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `socket_code` text null, `core_count` smallint, `base_clock_mhz` integer, `tdp_w` integer。 / 関連・索引: (`socket_code`)。

● gpu_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `chipset` text, `vram_gb` smallint, `card_length_mm` integer null CHECK >0。 / 関連・索引: (`card_length_mm`)。

● motherboard_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `socket_code` text null, `ddr_generation` text null CHECK DDR4/DDR5, `form_factor` text null CHECK ATX/mATX/ITX。 / 関連・索引: (`socket_code`, `ddr_generation`, `form_factor`)。

● memory_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `ddr_generation` text null, `capacity_gb` smallint, `module_count` smallint, `speed_mt_s` integer。 / 関連・索引: (`ddr_generation`, `capacity_gb`)。

● ssd_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `capacity_gb` integer, `interface` text, `form_factor` text。 / 関連・索引: (`capacity_gb`, `interface`)。

● psu_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `rated_w` integer, `form_factor` text, `efficiency_grade` text。 / 関連・索引: (`rated_w`)。

● case_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `max_gpu_length_mm` integer null, `outer_length_mm/width_mm/height_mm` integer, `supported_form_factors` text[] null。 / 関連・索引: フォームファクター検索はGIN索引。

● cooler_specs

主要カラム（型・制約）: `product_id` uuid PK FK, `supported_socket_codes` text[] null, `height_mm` integer, `cooling_type` text。 / **関連・索引:** 対応SocketにGIN索引。

カテゴリ別仕様は該当カテゴリの1表に1行のみ置く。公開時はその行の存在をDB関数で検証するが、互換性に必要な値の欠落自体はデータセット検証のため許す。その欠落は判定不能となる。画像は公開バケットに直接無制限公開せず、公開商品への関連を確認した経路で配信する。

4.3 在庫・カート・注文・決済

● inventory

主要カラム（型・制約）: `product_id` uuid PK FK, `on_hand` integer CHECK >=0, `allocated` integer CHECK >=0 AND <=on_hand, `version` bigint DEFAULT 0, `updated_at`。販売可能=on_hand-allocated。 / **関連・索引:** `allocated>0`部分索引。注文処理は行ロック。

● stock_allocations

主要カラム（型・制約）: `id` uuid PK, `order_id` uuid FK, `product_id` uuid FK, `quantity` integer CHECK >0, `state` text CHECK(active,consumed,released), `expires_at` timestamptz, `resolved_at` timestamptz null。 / **関連・索引:** (`order_id`,`product_id`)一意、(`state`,`expires_at`)。

● carts

主要カラム（型・制約）: `id` uuid PK, `user_id` uuid FK null, `anonymous_token_hash` text null, `created_at/updated_at`。所有者は片方のみ。 / **関連・索引:** `user_id`またはtoken hashに一意索引。匿名tokenはHttpOnly署名付きCookie。

● cart_items

主要カラム（型・制約）: `cart_id` uuid FK, `product_id` uuid FK, `quantity` integer CHECK 1..10, `updated_at`、複合PK。 / **関連・索引:** 商品FK索引。価格は保存せず都度取得。

● orders

主要カラム（型・制約）: `id` uuid PK, `user_id` uuid FK, `status` text CHECK(payment_pending,paid,payment_failed,expired,review_required), `currency` char(3) DEFAULT JPY, `goods_total_yen/shipping_base_yen/shipping_heavy_yen/shipping_total_yen/tax_total_yen/grand_total_yen` integer CHECK >=0, `tax_rate_basis_points` smallint, `shipping_rule_version` text, `origin_snapshot` jsonb, `address_snapshot` jsonb, `compatibility_snapshot` jsonb, `checkout_key` uuid unique, `created_at/updated_at/paid_at`。 / **関連・索引:** (`user_id`,`created_at` desc), (`status`,`created_at`)。金額の整合性はDB CHECKと注文関数で確認。

● order_items

主要カラム（型・制約）: `id` uuid PK, `order_id` uuid FK, `product_id` uuid FK null, `sku_snapshot/name_snapshot/brand_snapshot` text, `unit_price_yen` integer, `quantity` integer, `line_total_yen` integer, `tax_rate_basis_points` smallint, `spec_snapshot` jsonb, `weight_g_snapshot` integer, `pack_snapshot` jsonb。 / **関連・索引:** (`order_id`)、`product_id`。商品削除後も記録を残す。

● payment_attempts

主要カラム（型・制約）: `id` uuid PK, `order_id` uuid FK, `attempt_no` integer, `state` text CHECK(created,processing,succeeded,failed,expired,review_required), `stripe_session_id` text unique null, `stripe_payment_intent_id` text unique null, `amount_yen` integer, `expires_at` timestamptz, `created_at/updated_at`。 / **関連・索引:** (`order_id`,`attempt_no`)一意、未終了試行は注文ごと1件の部分ユニーク索引。

● payment_events

主要カラム（型・制約）: `stripe_event_id` text PK, `attempt_id` uuid FK null, `event_type` text, `received_at` timestamptz, `processed_at` timestamptz null, `outcome` text CHECK(processed,ignored,needs_review)。 / **関連・索引:** (`attempt_id`,`received_at`)。署名検証後に記録。

`inventory`の調整は管理関数を通し`inventory_adjustments(id,product_id,delta,reason,actor_id,created_at)`に残す。通常の注文取消しや返金は初期版の利用者機能ではなく、決済後の例外は管理者の確認対象とする。カート明細の上限10は操作単位の過大数量を抑える技術値で、置き置き上限とは無関係である。

4.4 設定・通知・監査

● shipping_settings

主要カラム（型・制約）: `id` uuid PK, `version` text unique, `origin_prefecture_code` smallint DEFAULT 13, `base_fee_yen` integer NOT NULL DEFAULT 940 CHECK >=0, `free_threshold_yen` integer DEFAULT 10000, `heavy_threshold_g` integer DEFAULT 20000, `heavy_rule_json` jsonb NOT NULL, `yamato_source_url` text, `source_checked_at` timestamptz, `active_from` timestamptz, `created_by` uuid。 / **関連・索引:** 有効版1件のみ。重量物料金表は公式情報に対応する発送元・配送先・サイズ・重量別の版管理データ。

● smtp_settings

主要カラム（型・制約）: id uuid PK, host text, port integer CHECK 1..65535, tls_mode text, sender_address text, sender_name text, username text, secret_ref text, is_active boolean, updated_by uuid, updated_at. / **関連・索引:** 有効設定1件。パスワードはVault等の保護領域、表には参照のみ。

● notification_jobs

主要カラム（型・制約）: id uuid PK, kind text, recipient_hash text, payload_ref text, state text, attempt_count smallint, next_attempt_at timestampz, created_at/updated_at. / **関連・索引:** (state,next_attempt_at)。メール本文・宛先の長期保存を避ける。

● audit_logs

主要カラム（型・制約）: id uuid PK, actor_id uuid null, action text, entity_type text, entity_id uuid null, change_summary jsonb, request_id text, created_at. / **関連・索引:** (entity_type,entity_id,created_at desc), (actor_id,created_at desc)。秘密・認証コード・住所本文を記録しない。

SMTP接続先を管理画面から変更する要件のため、設定値と秘密参照を分ける。Vercelの環境変数は固定のHook署名・Vault接続などに限定し、SMTP認証情報は管理者が更新できる保護領域に保存する。Vaultの復号権限は送信処理に限定する。設定の投入・変更は監査ログ対象である。

5. Supabase RLS・DB権限

公開スキーマの全テーブルでRLSを有効にし、anon/authenticatedへの既定権限を剥奪して必要なSELECTだけ付与する。管理者の更新・注文確定・在庫調整はVercelサーバーの権限検証後に、限定されたDB関数で実行する。サービスロールはRLSを迂回するためブラウザに渡さず、利用箇所とログを限定する。auth.uid()がnullの経路を許可しない。管理者判定関数は非公開スキーマに置き、SECURITY DEFINERの場合は固定search_pathと実行権限を設定する。

● 公開商品・仕様・画像

未ログイン: publishedのみ読取 / **会員:** 同左 / **管理者:** 下書き含む読取 / **書込経路:** 管理APIのみ

● profiles,addresses

未ログイン: 不可 / **会員:** user_id=auth.uid()のみ読取／住所・表示名更新 / **管理者:** 業務上必要な閲覧のみ / **書込経路:** 本人API／管理API

● carts,cart_items

未ログイン: 自分の匿名Cookieからサーバー経由 / **会員:** 自分のカートのみ / **管理者:** 原則閲覧不要 / **書込経路:** カートAPI

● orders,order_items,payment_attempts

未ログイン: 不可 / **会員:** orders.user_id=auth.uid()に従属する読取 / **管理者:** 管理画面で必要な読取 / **書込経路:** 注文・Webhook関数のみ

● inventory,stock_allocations

未ログイン: 公開可用数の投影のみ / **会員:** 同左 / **管理者:** 詳細読取 / **書込経路:** DB関数のみ

● admin_memberships,sms_challenges,shipping_settings,smtp_settings,notification_jobs,audit_logs

未ログイン: 不可 / **会員:** 原則不可。公開送料規則だけ投影 / **管理者:** 管理範囲のみ / **書込経路:** 限定関数／管理API

個人注文の詳細は注文所有者をEXISTSで確認するポリシーとし、明細や決済試行を直接ID推測で読めないようにする。管理者権限の付与・剥奪は公開管理画面の一般機能にはしない。Storageは管理者のみアップロード・削除し、公開状態に応じた配信経路を使う。RLS試験では匿名、会員A、会員B、管理者の許可・拒否、サービスロール誤使用、下書き商品漏えいを確認する。

6. API・サーバー処理契約

共通の応答は成功時{data, requestId}、失敗時{error:{code,message,fieldErrors?},requestId}。入力はZodで検証し、秘密やDB内部エラーを返さない。認証不在401、権限不足403、対象非公開404、形式不正400、状態競合409、再試行可能503を基本とする。状態変更はPOST/PATCH/DELETEのみ。CookieはHttpOnly/Secure/SameSite=Lax、認証済み変更はOrigin検査、管理・決済・認証にはレート制限を設ける。再送可能な注文開始にはIdempotency-Keyを要求する。

● GET /api/products

入力・検証: q最大100字、カテゴリ、用途、メーカー、価格整数範囲、仕様フィルタ、並べ替え、page。 / **権限・成功出力・主な失敗:** 公開。公開商品24件／ページ、総数、条件。無効条件400。

● POST /api/compatibility

入力・検証: カテゴリ別の商品ID、各カテゴリ最大1件。 / **権限・成功出力・主な失敗:** 公開。5判定のstatus/reason/comparedValues/matchingUrl。非公開ID404。

● GET/PUT /api/cart

入力・検証: PUTは商品ID・数量1〜10。 / 権限・成功出力・主な失敗: 匿名Cookieまたは会員本人。現在価格・可用数・概算送料。売切れ409。

● POST /api/cart/merge

入力・検証: 匿名Cookie。 / 権限・成功出力・主な失敗: ログイン後のみ。重複商品は数量合算、上限と在庫で調整し結果通知。

● POST /api/checkout/quote

入力・検証: 配送先IDまたは検証済み新住所。 / 権限・成功出力・主な失敗: 会員。再計算した商品・正式送料・税・合計・互換性。未設定送料503、在庫不足409。見積IDと期限を返す。

● POST /api/checkout/start

入力・検証: 見積ID、Idempotency-Key、利用者の確認済み状態。ブラウザの金額は受け取らない。 / 権限・成功出力・主な失敗: 会員。金額再検証→注文・引当→StripeテストSession URL。差異409、外部障害503。

● GET /api/checkout/status?orderId=

入力・検証: 注文ID。 / 権限・成功出力・主な失敗: 注文所有者のみ。状態、案内、再試行可否。

● POST /api/webhooks/stripe

入力・検証: 生リクエストとStripe署名。 / 権限・成功出力・主な失敗: 公開到達可能だが署名必須。受理200、署名不正400。イベントID重複は200で再処理しない。

● POST /api/internal/reconcile-payments

入力・検証: Supabase Cronからの署名付き呼出し、対象件数上限。 / 権限・成功出力・主な失敗: 内部署名のみ。Vercel側でStripe状態照合後、期限切れ試行と引当を確定。失敗時は次回再試行・管理通知。

● POST /api/auth/sms/start / verify

入力・検証: 目的、対象フロー、6桁コード。 / 権限・成功出力・主な失敗: 登録・再設定フローの短命トークン。発行・照合状態のみ返す。制限超過429。

● POST /api/auth/email-hook

入力・検証: Supabase Auth Hookの署名付きペイロード。 / 権限・成功出力・主な失敗: Hook署名検証、テンプレート限定、SMTP送信結果。署名不正401、送信失敗503。

● GET/POST/PATCH /api/account/addresses

入力・検証: 国内住所の各欄、既定フラグ。 / 権限・成功出力・主な失敗: 会員本人のみ。所有者違い404。

● DELETE /api/account/addresses/[id] / POST /api/account/delete

入力・検証: 住所ID／再認証済み削除確認。 / 権限・成功出力・主な失敗: 会員本人のみ。住所は即削除。アカウントは決済中処理を安全に終了してから本人データ・Authアカウントを削除し、幂等に再実行可能。

● /api/admin/products, /inventory, /orders, /settings/*

入力・検証: 各型付き入力、更新版番号。 / 権限・成功出力・主な失敗: 管理者のみ。更新結果と監査ID。競合409、秘密は応答しない。

Server Componentは商品表示のための取得をデータ層へ直接行い、注文・管理の変更はRoute Handlerへ集約する。外部通知の認証は会員Cookieではなく署名で行う。APIの操作ログにはrequestIdを付けるが、パスワード・OTP・カード・SMTP秘密・住所全文は出さない。

7. 商品検索・互換性・カート

検索は公開商品に限定し、キーワードの正規化、パラメータ化した全文／部分一致、カテゴリ・用途・メーカー・価格・仕様のAND条件、価格または新着の安定ソート（最後にID）を行う。0件の条件は消さず、解除候補を示す。商品数増加時に外部検索へ切り出せるよう検索結果の契約を固定する。管理者の下書き検索は別経路とする。

互換性は次の順に評価し、比較する2品の片方がないときはnot_applicable、必要仕様がnullならunknown、値が異なればincompatible、一致すればcompatible。①CPU.socket=MB.socket、②MB.DDR=メモリ.DDR、③MB.form_factor∈ケース.supported_form_factors、④GPU.card_length_mm≤ケース.max_gpu_length_mm、⑤CPU.socket∈クーラー.supported_socket_codes。理由には比較値と不足値を含める。incompatibleでは対応規格をクエリにした商品一覧リンクを生成する。仕様欠損を互換とみなさない。警告は構成確認と注文確認に表示し、カート投入と購入を妨げない。

カートは匿名Cookieをサーバーでハッシュ照合し、ログイン時に会員カートへ統合する。商品価格をカートに固定しない。表示時は最新税込単価・販売可能数・概算送料を読む。数量更新は可用数より多ければ409で最新値を返す。正式注文時は価格、公開状態、在庫、送料をすべて再評価する。ログアウト後、会員カートは本人以外に見せない。

8. 送料・税・見積・スナップショット

送料規則是版を持ち、東京都発送を初期値とするが管理設定で変更可能。全国一律の通常送料B=940円（税込）を基礎に、商品税込小計 $G \geq 10000$ 円なら通常送料を0円とする。北海道・沖縄県を含め、離島という理由だけの加算はしない。20kg以上の商品ごとに、ヤマト運輸公式表に基づく発送元・届け先・梱包サイズ・重量の運賃を重量物追加送料H_iとし、送料無料でも加算する。初期版は商品1点を1梱包として各点の運賃を合算する。合計送料 $S = (G \geq 10000 ? 0 : 940) + \sum H_i$ 。重量物の料金表は管理者が公式情報の出典・確認日とともに版管理し、見積時に該当行を引く。商品の荷姿・サイズ・重量が未登録、料金表に該当がない場合はSHIPPING_UNAVAILABLEとし、金額を仮定せず決済開始を止める。ヤマト運輸の取扱上限を超える商品は初期版で登録・公開できない。カートでは配送先未確定の「概算」、注文確認では配送先確定後の「確定」を表示し、根拠となる公式ページと料金表確認日を示す。

税込価格を基礎に、初期版は標準税率10%のみを扱う。商品小計 $G = \sum(\text{単価} \times \text{数量})$ 、送料S、税込総額 $T = G + S$ 、内税額は税率ごとに $\text{floor}(T \times 10 / 110)$ で1回計算し、商品と送料の税率が将来異なる場合は税率別に集計する。端数は切り捨てで固定し、明細ごとの税額丸めを合算しない。注文作成時に商品名・SKU・仕様・税込単価・数量・税率・重量・梱包、住所、発送元、送料内訳、料金規則版、互換性判定をスナップショットとして保存する。後日の商品・住所・設定変更で既存注文を再計算しない。Stripeへ送る額はこの確定総額と完全一致させる。

9. 注文・在庫引当・決済の状態機械

注文開始は(1)会員・メール/SMS条件・住所確認、(2)カート再読込と現在価格・公開状態・可用数検証、(3)送料・税・互換性再計算、(4)DBトランザクション内で商品ID順にinventory行ロック、on_hand-allocated>=quantity確認、allocated増加、注文・明細・引当・決済試行作成、(5)Stripe Checkout Session生成、(6)Session ID保存と遷移URL返却、の順。checkout_keyとStripe Idempotency-Keyは同一購入操作で固定し、再送しても別注文を作らない。Stripe作成に失敗した場合は試行失敗と引当解放をDBで原子的に記録する。DB保存が応答不明なら同一キーで照会してから再試行する。

● 注文

状態遷移・条件: payment_pending→paidは署名済み成功イベントと金額・通貨・Session照合時のみ。→payment_failed/expiredは確定失敗・期限切れ。矛盾・遅延成功はreview_required。

● 決済試行

状態遷移・条件: created→processing→succeeded/failed/expired。再試行は新規試行番号を作る。同じ注文の成功は1回だけ。

● 引当

状態遷移・条件: active→consumedでon_handとallocatedを共に数量分減算。active→releasedでallocatedのみ減算。二重遷移は無操作。

Webhookは生ボディの署名と許容時刻を検証し、payment_events.stripe_event_idで重複排除する。受信順に依存せずStripe Session/PaymentIntentの現在状態を照合し、成功は金額・JPY・注文IDメタデータ・試行IDを検証する。成功処理と在庫消費を同一DBトランザクションにする。失敗・期限切れも引当解放と状態変更を同一トランザクションにする。Stripeからの応答だけでDB処理が失敗したときはWebhookを非2xxとして再送に任せ、補償ジョブも照合する。重複通知は200で応答し追加処理しない。画面の戻りURLは結果表示の契機に限る。

Sessionの有効期限と引当期限は同じ設定値に揃え、初期設計値30分とする。ただし期限時刻直後に機械的に解放せず、Supabase Cronが署名付きVercel内部APIを起動し、Vercel側でStripeの現在状態を確認してから解放する。Stripe秘密鍵をDB Cronに渡さない。外部照合不能なら引当を維持してreview_requiredで管理者へ通知し、二重販売を避ける。期限後の成功通知は在庫の消費状況を再確認し、安全に確定できる場合だけpaidへ進める。既に解放・再販売済みなら自動でpaidにせずreview_requiredとし、管理者がStripe側状況を確認する。失敗後の再試行では価格・送料・在庫を再見積り、必要なら新注文とする。注文完了メールはpaid確定後に1回だけキューへ投入する。

在庫調整・注文引当は管理画面からの直接SQL更新を禁止し、DB関数でロックと監査を行う。決済失敗・在庫不足・Webhook遅延のデモ用シナリオはテストデータとStripeテスト手段で再現し、一般利用者に内部操作を露出しない。

10. 認証・Google・メール・SMS

メール+パスワード登録はSupabase Authで開始し、メールリンク確認と6桁SMSコード照合の両方が終わるまで注文開始不可とする。確認順序はどちらが先でもよい。公開デモのSMSは本人電話番号へ送らず、ログイン済みの当該フロー画面内「デモ通知」に短時間だけ表示する。コード自体はDBに平文保存せず、ハッシュ・目的・期限・試行回数を保持する。Google登録/ログインはSupabase AuthのOAuthを使い、スコープはopenid email profileの必要最小限とし、GoogleトークンをアプリDBに保存しない。Google利用者にSMSは要求しない。配送先はサイト内で入力する。

パスワード再設定はSupabaseのメールリンクを起点に、同一フローで6桁SMS照合後に新パスワードを確定する。通常ログインにSMSは不要。コード有効10分、試行5回、再送間隔60秒、発行回数に時間制限を設ける。メール送信・SMS発行の応答は存在するアカウントか推測できない表現にする。Development/Testだけメール・SMS確認省略フラグを許可し、ビルド時・起動時にNODE_ENV=productionまたは

Vercel Productionで有効なら起動を失敗させる。公開デモの模擬SMSはこの省略機能とは別で、コード入力を必須とする。

Supabase Auth Send Email Hookは署名を検証するVercel経路で受け、許可されたテンプレート（確認、再設定など）だけをSMTPで送る。SMTP設定は管理者がホスト・ポート・TLS・送信元を更新し、秘密は更新時のみ受け付ける。接続試験は管理者操作で検証先へ送り、結果と時刻を記録する。設定に失敗したら旧設定を維持し、秘密を画面へ戻さない。認証メールが送れない場合は登録・再設定を未完了のまま保ち、再送導線と管理者向け障害表示を出す。注文通知は再試行可能なジョブで送る。実SMSゲートウェイは開発・テストの自己端末検証専用の差替え境界とし、提供会社・費用を未確定とする。

11. セキュリティ・個人情報・監査

TLSを必須にし、CSP、クリックジャッキング防止、適切なCookie属性を設定する。管理APIは認証・[admin_memberships](#)・Origin・レート制限を毎回確認する。権限付与はDB管理手順で監査し、通常の管理UIには置かない。入力はサーバーで型・文字数・許可値を検証し、HTMLとして再解釈しない。SQLはパラメータ化する。商品画像は種類・容量・寸法を検査し、推測可能な管理用パスをそのまま公開しない。

秘密はVercel環境別秘密変数またはSupabase Vault等に置き、GitHub、ブラウザ、ログ、監査差分に含めない。Stripe鍵はテスト鍵のみ。Preview/ProductionでSupabase、Stripeテストデータ、SMTP宛先、Hook署名を分離する。注文・住所は本人と許可された管理者のみ閲覧。管理者一覧には必要な注文情報だけ表示し、個人情報の大量出力機能を初期版で設けない。監査対象は商品公開・価格・在庫調整・送料・発送元・SMTP設定・管理者権限・例外注文処理。監査ログは追記専用とし、秘密と住所本文は記録しない。

公開デモの配達先は自由入力とし、架空情報の入力を案内する。規約・個人情報案内には実販売・実課金・実発送なし、メール実送信、SMS模擬、保存する情報と利用目的、30日以内の自動削除、本人による住所・アカウント削除を明示する。住所は本人操作で即時削除する。アカウント削除は再認証後、決済中の試行を照合・終了し、住所・カート・注文・通知情報・プロフィール・Authアカウントを削除する。削除処理が途中失敗した場合は再実行可能なジョブで完了させる。登録から30日を経過したアカウントは同じ手順で自動削除し、注文は作成から30日を超えて保持しない。15分間隔の削除ジョブで期限到来分を処理し、失敗時は通知して再試行する。削除前に保留中の決済があればStripeのテスト状態を照合する。Stripeには住所・氏名・メールなどの個人情報をメタデータとして渡さず、注文IDだけを渡す。監査ログも個人情報本文を含めず、削除済み利用者と紐づけられない形にする。バックアップにも30日方針を満たせる保持設定を公開前に確認し、満たせない構成では個人情報を含むバックアップを作らない。

12. テスト設計

● Vitest 単体

必須ケース: 5種の互換／不一致／欠損、検索条件、カート統合、送料無料境界9,999/10,000円、20kg境界、税丸め、金額スナップショット、状態遷移拒否。

● DB統合

必須ケース: 2会員同時購入の残数1、商品ID順ロック、引当失敗時の全ロールバック、失敗・期限切れの一回解放、遅延成功、同一注文二重成功防止、管理在庫調整競合。

● RLS

必須ケース: 匿名／会員A／会員B／管理者の各表・Storage許可拒否、本人注文の明細間接参照、下書き商品漏出、サービスロールをクライアントに使わない検査。

● API統合

必須ケース: Zod境界、401/403/404/409/429/503、幂等キー再送、Stripe署名不正・重複・逆順・金額不一致、メールHook署名、SMTP障害。

● データ削除

必須ケース: 本人の住所削除、再認証後のアカウント削除、30日期限、処理途中失敗からの再実行、削除後RLSでの不在、バックアップ保持条件。

● Playwright

必須ケース: PC・タブレット・スマホで探索→構成→カート→会員→住所→正式見積→Stripeテスト成功／失敗→履歴、Google OAuthテスト、模擬SMS、在庫切れ、互換不一致／不明、管理画面。

● 非機能

必須ケース: 320px・200%拡大、キーボード、コントラスト、見出し、入力エラー、主要画面のWeb Vitals、秘密漏えい・ログ確認。

テスト用商品は互換・不一致・仕様不足・売切れ・残数1・重量境界を含む。初期商品件数は要件定義どおり固定しない。外部サービスを使う自動試験は専用のテスト資格情報を使い、実メール誤送信・実SMS誤送信を防ぐ。性能の初期目標は主要公開画面のLCP 2.5秒以内、CLS 0.1以下、INP 200ms以下（通常の計測条件で評価）とし、実測で改善する。

13. CI/CD・環境・運用

GitHub PRで形式検査、TypeScript型検査、Vitest、DBスキーマ検証、RLSテスト、ビルド、Playwrightの主要導線を実行する。Vercel Git連携でPRごとPreview、main反映でProduction。Previewは専用Supabase、Stripeテスト環境、送信先制限付きSMTP、模擬SMSを使う。ProductionもStripeテスト鍵のみで、ライブ鍵の存在をデプロイ検査で拒否する。環境変数はVercelの環境別スコープで管理し、GitHub Actionsには必要最小限の短命権限または環境秘密を付与する。

DB変更はバージョン付きマイグレーションでレビューし、追加→データ補完→切替→旧列廃止の順に公開する。破壊的変更前にバックアップ・復元手順を検証する。デプロイ後に商品探索、ログイン、見積、Webhook疎通、管理画面をスモーク確認する。障害監視は注文失敗率、Webhook滞留、期限切れ引当、SMTP失敗、未処理[review_required](#)、30日削除ジョブの失敗を対象にする。個人情報を含まないメトリクスとリクエストIDで追跡する。

14. 基本設計からの引き継ぎ確認

● 画面の入力・エラー・空状態、遷移、UI部品

本書: 第2・3章

● テーブル・制約・索引・RLS・監査・初期データ

本書: 第4・5・12章

● 引当期限、決済遅延・重複・失敗・復旧

本書: 第9章

● 送料式、税端数、注文スナップショット

本書: 第8章

● SMTP権限・秘密、模擬SMS、通知、送信制限

本書: 第10・11章

● 公開デモ規約、個人情報、削除

本書: 第11・13章。利用者確認済み

● レスポンシブ、アクセシビリティ、性能、監視・障害

本書: 第2・12・13章

15. 詳細設計後の実装向け成果物

実装着手時に、この設計を元に画面ワイヤー、Zodスキーマ、SQLマイグレーション、RLSポリシーSQL、API OpenAPI記述、メール文面、テストデータ定義、運用手順を作成する。本書のテーブル・API名は設計上の契約であり、現時点でコードやDBは作成していない。将来の構成保存は[builds](#)と[build_items](#)を追加し、第7章の互換性判定サービスを再利用する。置き置き・実店舗受取は在庫引当とは別の業務状態で追加する。

16. 利用開始前に判断を要する事項

送料と公開デモの個人情報方針に関する質問は利用者が承認済みである。実装のための追加質問はない。

外部サービス費用：Vercel、Supabase、SMTP送信先、実端末SMS検証先の有料プラン・費用上限。現時点で契約・課金を伴う操作を行わない。有料利用が必要になれば、サービス名・見積額・無料代替案を示して導入前に承認を得る。

17. 参照した公式資料

- [Next.js App Router](#)、[Supabase SSR](#)、[Supabase RLS](#)、[Supabase Storage権限](#)、[Supabase Cron](#)
- [Supabase Google OAuth](#)、[Send Email Hook](#)、[Vault](#)
- [Stripe Checkout](#)、[テスト環境](#)、[幂等性](#)、[Webhook](#)
- [Vercel Git連携](#)、[環境変数](#)、[SMTP](#)
- [ヤマト運輸料金表](#)、[宅急便サイズ](#)、[離島料金の案内](#)、[国税庁 消費税率](#)